

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Кафедра інженерії програмного забезпечення в енергетиці

КУРСОВА РОБОТА

з дисципліни:

«Основи Веб-програмування»

тема: **«Додаток, який створює сильні та безпечні паролі.»**

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Кафедра інженерії програмного забезпечення в енергетиці

КУРСОВА РОБОТА

з дисципліни: «Основи Веб-програмування»
(назва дисципліни)

на тему: «Додаток, який створює сильні та безпечні паролі.»

Студент 2 курсу групи ТВ-43
напряму підготовки **121 Інженерія програмного забезпечення**
Білоус А.О.
(прізвище та ініціали)

GitHub репозиторій [посилання](#)

Керівник д.т.н., доцент, Недашківський О. Л.
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна оцінка _____
Кількість балів: _____ Оцінка: ECTS _____

Член комісії

(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

Київ – 2026 рік

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”

Навчально-науковий інститут атомної та теплової енергетики

(повна назва)

Кафедра інженерії програмного забезпечення в енергетиці

(повна назва)

Напрямок підготовки 121 Інженерія програмного забезпечення

(шифр і назва)

З А В Д А Н Н Я
НА КУРСОВУ РОБОТУ СТУДЕНТУ

Білоус Андрій Олегович

(прізвище, ім'я, по батькові)

1. Тема роботи – «Додаток, який створює сильні та безпечні паролі.»

Керівник курсової роботи – Недашківський О.Л.

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

2. Строк подання студентом роботи: «9» травня 2026 р.

3. Вихідні дані до проекту (роботи): мова програмування – Python, фреймворк – Django, дизайн – HTML, CSS, Bootstrap 5.

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити): розробити веб-додаток менеджер паролів мовою програмування Python.

5. Дата видачі завдання: 22 лютого 2026

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів виконання дипломного проекту (роботи)	Строк виконання етапів проекту (роботи)	Примітка
1.	Затвердження теми роботи	22.02.2026	
2.	Вивчення та аналіз задачі	23.02.2026	
3.	Проектування архітектури та бази даних	20.03.2026	
4.	Створення тест плану	15.02.2026	
5.	Розробка програмного коду	20.02.2026	
6.	Розробка інтерфейсу програми	18.03.2026	
7.	Тестування програми	20.03.2026	
8.	Оформлення пояснювальної записки	06.03.2026	

Студент

(підпис)

Білоус А.О.

(прізвище та ініціали)

Керівник курсової роботи

(підпис)

Недашківський О. Л.

(прізвище та ініціали)

АНОТАЦІЯ

У ході виконання курсової роботи було освоєно основи розробки безпечних веб-додатків за допомогою фреймворку Django та мови програмування Python. Отримані знання було використано для реалізації проекту веб-орієнтованого менеджера паролів. Під час виконання роботи увага приділялась кожному етапу розробки програмного забезпечення - від постановки завдання та проектування бази даних до втілення готового рішення з привабливим користувацьким інтерфейсом. Основною метою роботи було розроблення функціоналу додатку, який би допоміг користувачеві генерувати надійні паролі, перевіряти їх на стійкість до зламу та безпечно зберігати у зашифрованому вигляді в особистому сховищі. Обсяг пояснювальної записки 26 аркушів, кількість ілюстрацій 13, 3 додатки.

ANNOTATION

During the course work, the basics of developing secure web applications using the Django framework and the Python programming language were mastered. The acquired knowledge was used to implement a web-based password manager project. During the work, attention was paid to each stage of software development - from task formulation and database design to implementing a ready-made solution with an attractive user interface. The main goal of the work was to develop application functionality that would help the user generate strong passwords, check their resistance to hacking, and securely store them in an encrypted form in a personal vault. The explanatory note is 26 pages long, the number of illustrations is 13, and there are 3 appendices.

ЗМІСТ

<i>ВСТУП</i>	<i>6</i>
<i>РОЗДІЛ 1. ОПИС ПРОЕКТУ</i>	<i>7</i>
1.1 Структура та основні вимоги	8
1.1.1. Бібліотеки та інструменти	9
1.1.2. Структура додатку	9
1.1.3. Реалізація функцій	10
<i>РОЗДІЛ 2. ПРОЦЕС РОЗРОБКИ ДОДАТКУ</i>	<i>11</i>
2.1 Загальні поняття про технологію	11
2.2 База даних та обробка даних	12
2.3 Переваги використання обраних технологій	13
2.4 Опис папок проекту	13
2.5 Ключові файли проекту	14
<i>РОЗДІЛ 3. ХІД ВИКОНАННЯ РОБОТИ</i>	<i>17</i>
<i>РОЗДІЛ 4. КЕРІВНИЦТВО КОРИСТУВАЧА</i>	<i>21</i>
4.1. Екран авторизації	21
4.2. Головний екран (Генератор паролів)	22
4.3 Збереження пароля та Особисте сховище	23
4.4 Перевірка надійності пароля	25
4.5 Управління профілем	25
4.6 Панель адміністратора	26
<i>ВИСНОВКИ</i>	<i>27</i>
<i>СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ</i>	<i>29</i>
<i>ДОДАТОК 1</i>	<i>30</i>
<i>ДОДАТОК 2</i>	<i>33</i>
<i>ДОДАТОК 3</i>	<i>36</i>

ВСТУП

У сучасному цифровому світі, де кількість онлайн-сервісів стрімко зростає, питання кібербезпеки та захисту персональних даних виступає на перший план. Використання слабких або однакових паролів для різних платформ є однією з головних причин витоку конфіденційної інформації. З метою вирішення цієї проблеми та оптимізації процесу управління обліковими даними, було створено веб-додаток - менеджер паролів PassGuard. Цей додаток, розроблений з використанням сучасних веб-технологій та криптографічних алгоритмів, забезпечує користувачів надійним інструментом для створення та збереження паролів. Основна функція додатку - допомогти користувачеві генерувати криптографічно стійкі паролі заданої довжини та складності в один клік, а також надати безпечне "Сховище" для їх зберігання. Модуль авторизації додатку включає реєстрацію користувачів, безпечний вхід в систему та управління особистим профілем. Крім того, додаток містить інтерактивний інструмент для перевірки надійності вже існуючих паролів користувача. Актуальність цього додатку полягає у зростаючій потребі захисту цифрової ідентичності. Наявність єдиного, захищеного майстер-паролем центру управління всіма обліковими записами робить цей додаток незамінним помічником для будь-кого, хто піклується про свою інформаційну безпеку.

РОЗДІЛ 1. ОПИС ПРОЕКТУ

Ця курсова робота має на меті розробку веб-додатку (менеджера паролів), який допомагає користувачам створювати надійні облікові дані та безпечно зберігати їх. Основна ідея проекту полягає у створенні зручного та інтуїтивно зрозумілого інструменту, який знімає з користувача необхідність запам'ятовувати десятки складних комбінацій для різних сайтів. Додаток використовує реляційну базу даних для збереження інформації та криптографічні механізми для її захисту. Особлива увага приділяється модулю автентифікації, що забезпечує високий рівень безпеки та ізоляцію даних різних користувачів один від одного. Основний функціонал додатку включає генерацію паролів за заданими параметрами (довжина, наявність спецсимволів, цифр тощо), збереження згенерованих даних із прив'язкою до конкретного сервісу, а також перегляд збережених записів в особистому кабінеті. Метою даного проекту є створення не лише функціонального, але й стійкого до вразливостей веб-додатку, який відповідає сучасним стандартам веб-розробки та сприяє підвищенню рівня цифрової гігієни користувачів.

1.1 Структура та основні вимоги

Основні вимоги додатку:

1. Авторизація та безпека:

- Реєстрація нових користувачів;
- Авторизація вже створених користувачів;
- Зміна облікових даних (логін, пошта, пароль) в особистому кабінеті;
- Ізоляція сесій (користувач бачить лише свої паролі).

2. Генератор паролів:

- Створення випадкових паролів;
- Налаштування довжини (від 8 до 32 символів);
- Вибір набору символів (великі літери, цифри, спеціальні символи).

3. Сховище (Особистий кабінет):

- Збереження згенерованого пароля разом із назвою сервісу та логіном;
- Перегляд списку збережених записів;
- Відображення статистики користувача.

4. Перевірка надійності:

- Інтерактивна перевірка складності введеного пароля у реальному часі.

5. Користувацький інтерфейс:

- Інтуїтивний та адаптивний інтерфейс (підтримка мобільних та десктопних пристроїв);
- Візуально привабливий дизайн з використанням сучасних UI-компонентів.

6. Адміністрування системи:

- Спеціальний доступ для суперкористувача (адміністратора);
- Панель керування всіма обліковими записами користувачів;
- Можливість примусової зміни електронної пошти та майстер-пароля користувача адміністратором.

1.1.1. Бібліотеки та інструменти

Для реалізації додатку будуть використовуватися наступні бібліотеки та інструменти:

- **Python** – високорівнева мова програмування, на якій написана вся серверна логіка.
- **Django** – потужний веб-фреймворк для мови Python, який забезпечує архітектуру MVT (Model-View-Template) та має вбудовані механізми безпеки і ORM.
- **PostgreSQL / SQLite** – реляційна система керування базами даних для надійного збереження моделей користувачів та їхніх записів.
- **Bootstrap 5** – популярний CSS-фреймворк, який використовується для створення адаптивного та стильного фронтенд-інтерфейсу (кнопки, форми, сітка).
- **JavaScript** – мова програмування, що використовується на стороні клієнта для динамічної генерації паролів без перезавантаження сторінки та розрахунку складності.

1.1.2. Структура додатку

Додаток розроблено за монолітною архітектурою та складається з наступних ключових компонентів:

- **Модуль маршрутизації (urls.py):** Відповідає за обробку вхідних URL-адрес та направлення запитів до відповідних контролерів.
- **Контролери (views.py):**
 - о `home_view` - обробляє сторінку генератора паролів.
 - о `register_view` / `login_view` / `logout_view` - функції для керування доступом.
 - о `save_password_view` - обробляє збереження нового запису в базу.
 - о `passwords_view` - формує сторінку Сховища користувача.
 - о `profile_view` - керує особистим кабінетом та зміною даних.
 - о `check_view` - відображає інструмент перевірки надійності.

- o `custom_admin_view` - контролер панелі адміністратора, який забезпечує керування базою користувачів.
- Моделі даних (`models.py`):
 - o `User` - стандартна модель Django для представлення користувача додатку.
 - o `PasswordRecord` - сутність збереженого пароля (містить поля: назва сервісу, логін, зашифрований пароль, зовнішній ключ на користувача).
- Шаблони (`Templates`):
 - o `base.html` - основний каркас сайту (шапка, бокове меню).
 - o `home.html`, `login.html`, `profile.html` тощо - сторінки, що наслідують базовий каркас.
 - o `custom_admin.html` - інтерфейс панелі керування користувачами для адміністратора.

1.1.3 Реалізація функцій

Авторизація: Система перевіряє унікальність логіна при реєстрації. При вході в систему паролі порівнюються у хешованому вигляді. Доступ до всіх внутрішніх функцій захищений декоратором `@login_required`.

Генерація паролів: Реалізована на стороні клієнта за допомогою JavaScript. Алгоритм формує рядок із заданої кількості випадкових символів обраних категорій (ASCII-символи).

Збереження даних: Після натискання кнопки збереження, дані через HTTP POST-запит відправляються на сервер, де створюється новий об'єкт `PasswordRecord`.

Відображення сховища: Сервер робить вибірку з БД виключно тих записів, зовнішній ключ (`user_id`) яких співпадає з ідентифікатором поточної сесії.

РОЗДІЛ 2. ПРОЦЕС РОЗРОБКИ ДОДАТКУ

Процес виконання даної курсової роботи включав у себе послідовні етапи, спрямовані на створення безпечного та функціонального веб-додатку для генерації та збереження паролів. Кожен етап був уважно розроблений та виконаний з метою забезпечення оптимальної роботи програми, гарантії безпеки даних та задоволення потреб користувача.

2.1 Загальні поняття про технологію

Python - високорівнева мова програмування загального призначення, яка швидко завойовує популярність серед розробників завдяки своїй простоті, читабельності та потужності. Мова призначена для розробки веб-додатків, аналізу даних, штучного інтелекту та автоматизації. Однією з ключових переваг Python є високий рівень виразності, строга динамічна типізація і наявність величезної стандартної бібліотеки, що робить її привабливим вибором для проектів, пов'язаних із кібербезпекою та обробкою даних.

Однією з найважливіших особливостей Python є її філософія, яка наголошує на продуктивності розробника. Читабельний синтаксис дозволяє створювати надійні та підтримувані системи з меншою кількістю рядків коду порівняно з іншими мовами. Крім того, наявність потужних криптографічних бібліотек робить Python ідеальним інструментом для розробки менеджерів паролів.

Django - це високорівневий веб-фреймворк для Python, який сприяє швидкій розробці та чистому, прагматичному дизайну. Створений досвідченими розробниками, він бере на себе багато рутинних завдань веб-розробки, дозволяючи зосередитися на написанні самого додатку без необхідності "винаходити колесо". Django дотримується принципу "включено батарейки"

Основні функції та інструменти Django:

- **ORM (Об'єктно-реляційне відображення):** Дозволяє взаємодіяти з базою даних за допомогою коду Python, без необхідності писати складні SQL-запити вручну.
- **Вбудована панель адміністратора:** Автоматично генерований, готовий до використання інтерфейс для управління даними (користувачами, паролями) одразу після створення моделей.
- **Високий рівень безпеки:** Django автоматично захищає додаток від найпоширеніших загроз, таких як SQL-ін'єкції, міжсайтовий скриптинг (XSS) та підробка міжсайтових запитів (CSRF).
- **Система маршрутизації (URL Dispatcher):** Гнучка система підключення веб-адрес до відповідних функцій-контролерів.
- **Шаблонізатор (Template System):** Потужна система для динамічної генерації HTML-сторінок із підтримкою наслідування шаблонів (наприклад, базового каркасу сайту).
- **Управління сесіями та авторизацією:** Готові модулі для реєстрації, логіну та перевірки прав доступу користувачів.

2.2 База даних та обробка даних

Для збереження конфіденційної інформації (моделей користувачів та їхніх паролів) у додатку використовується реляційна система керування базами даних (СКБД). На етапі розробки використовується вбудована SQLite, яка завдяки архітектурі Django ORM може бути легко замінена на потужну PostgreSQL під час розгортання на сервері без зміни жодного рядка бізнес-логіки.

База даних використовується для збереження двох основних сутностей:

- **User:** Вбудована модель, що містить захешовані паролі доступу до системи, імейли та логіни користувачів.
- **PasswordRecord:** Створена колекція записів, яка містить інформацію про цільовий сервіс, логін та сам згенерований пароль.

2.3 Переваги використання обраних технологій

Використання стеку Python + Django у розробці нашого менеджера паролів виявилось надзвичайно вигідним. Було виділено наступний перелік переваг:

- Безпека з коробки:

Вбудований захист за допомогою `{% csrf_token %}` та автоматичне хешування паролів користувачів алгоритмом PBKDF2 запобігають більшості кібератак.

- Ізоляція даних:

Завдяки системі сесій Django, кожен авторизований користувач має доступ виключно до свого особистого сховища збережених паролів.

- Швидкість розробки:

Наявність готової системи автентифікації та адмін-панелі дозволила зекономити час і зосередитися на бізнес-логіці (генераторі та перевірці надійності).

- Масштабованість:

Архітектура MVT дозволяє легко додавати нові функції (наприклад, темну тему чи зміну профілю) без порушення роботи існуючого функціоналу.

2.4 Опис папок проекту

Основна папка проекту містить усі файли та ресурси, пов'язані з розробкою веб-додатку. Структура відображає внутрішню організацію архітектури Django, розміщуючи різні компоненти у відповідних директоріях.

Основні папки включають:

- `core/`: Головна конфігураційна директорія проекту. Містить файли налаштувань підключення до бази даних, параметри безпеки та головний маршрутизатор URL-адрес.
- `manager/`: Директорія самого додатку (Application). Тут зберігається вся бізнес-логіка: контролери (`views.py`), моделі бази даних (`models.py`) та налаштування адмін-панелі.
- `manager/migrations/`: Папка, яка автоматично генерується Django. Містить історію змін структури бази даних (міграції).
- `manager/templates/manager/`: Папка з HTML-шаблонами. Містить файли користувацького інтерфейсу (`base.html`, `home.html`, `login.html`), які відображаються в браузері.
- `venv/`: Директорія віртуального середовища, де зберігаються всі встановлені бібліотеки та залежності Python, ізольовані від глобальної системи.

2.5 Ключові файли проекту

settings.py - головний файл конфігурації. У ньому підключаються створені додатки, налаштовується база даних (DATABASES), вказуються шляхи до статичних файлів та параметри безпеки (SECRET_KEY).

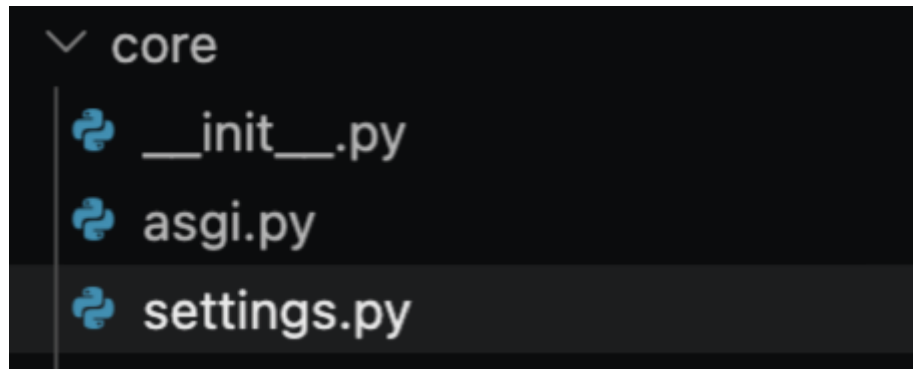


Рис. 2.1 – Файл settings.py

urls.py - файл маршрутизації проекту. Відповідає за зв'язок між URL-адресою, яку вводить користувач (наприклад, /register/), та конкретною функцією у views.py.

```
core > urls.py > ...
1  from django.contrib import admin
2  from django.urls import path
3  from manager import views
4
5  urlpatterns = [
6      path('admin/', admin.site.urls),
7      path('', views.home_view, name='home'),
8      path('login/', views.login_view, name='login'),
9      path('register/', views.register_view, name='register'),
10     path('logout/', views.logout_view, name='logout'),
11     path('save-password/', views.save_password_view, name='save_password'),
12     path('passwords/', views.passwords_view, name='passwords'),
13     path('check/', views.check_view, name='check'),
14     path('profile/', views.profile_view, name='profile'),
15     path('about/', views.about_view, name='about'),
16 ]
17
```

Рис. 2.2 – Файл urls.py

models.py - файл, де за допомогою класів Python описується структура таблиць бази даних. Тут реалізовано сутність PasswordRecord зі зв'язком ForeignKey до конкретного користувача.

```

manager > models.py > PasswordRecord > __str__
1  from django.db import models
2  from django.contrib.auth.models import User
3
4  class Category(models.Model):
5      name = models.CharField(max_length=100, verbose_name="Назва категорії")
6      user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='categories')
7
8      def __str__(self):
9          return f"{self.name} ({self.user.username})"
10
11 class PasswordRecord(models.Model):
12     service_name = models.CharField(max_length=255, verbose_name="Назва сервісу (напр. Google)")
13     login = models.CharField(max_length=255, verbose_name="Логін або Email")
14     encrypted_password = models.CharField(max_length=500, verbose_name="Зашифрований пароль")
15
16     category = models.ForeignKey(Category, on_delete=models.SET_NULL, null=True, blank=True, verbose_name="Категорія")
17     user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='passwords', verbose_name="Власник")
18     created_at = models.DateTimeField(auto_now_add=True)
19
20     def __str__(self):
21         return f"{self.service_name} - {self.login}"

```

Рис. 2.3 – Файл model.py

views.py - ключовий файл бізнес-логіки (контролерів). Тут розміщені функції для реєстрації, входу в систему, обробки форм збереження нових паролів та виведення їх у Сховищі.

```

def login_view(request):
    error_message = None
    if request.method == 'POST':
        user_name = request.POST.get('username')
        pass1 = request.POST.get('password')

        user = authenticate(request, username=user_name, password=pass1)

        if user is not None:
            login(request, user)
            return redirect('home')
        else:
            error_message = "Невірний логін або пароль!"

    return render(request, 'manager/login.html', {'error': error_message})

```

Рис. 2.4 – Файл views.py

base.html - основний файл-каркас графічного інтерфейсу. Містить підключення бібліотеки Bootstrap 5 та навігаційне меню. Слугує основою для всіх інших сторінок додатку.

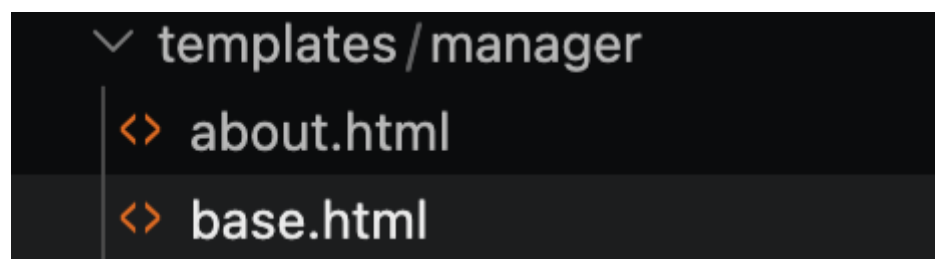


Рис 2.5 – Файл base.html

РОЗДІЛ 3. ХІД ВИКОНАННЯ РОБОТИ

Маршрутизація (urls.py)

Вхідною точкою в логіку програми є файл `urls.py` (URL Dispatcher), який відповідає за маршрутизацію. Цей модуль отримує всі вхідні HTTP-запити від клієнта та направляє їх до відповідних функцій-контролерів у файлі `views.py`. Для кожної сторінки (генератор, реєстрація, вхід, кабінет, збереження паролів) прописаний свій унікальний шлях (path), що забезпечує чітку структуру веб-додатку та зручну навігацію.

Data Models (models.py)

До основних моделей веб-додатку належать `User` та `PasswordRecord`. Модель `User` є вбудованою сутністю Django, яка відповідає за безпечне збереження логіну, електронної пошти та захешованого майстер-пароля користувача. Сутність `PasswordRecord` відображає збережені користувачем паролі, які записуються в базу даних SQLite (з подальшим масштабуванням до PostgreSQL).

Для кожного збереженого запису прив'язується зовнішній ключ (ForeignKey), який вказує на конкретного користувача-власника. Це гарантує, що жоден користувач не зможе отримати доступ до паролів іншого. Сутність містить такі поля:

- `user`: зв'язок із моделлю `User`.
- `service_name`: назва ресурсу (наприклад, Facebook).
- `login`: логін або пошта для цього ресурсу.
- `encrypted_password`: сам згенерований пароль, який зберігається в базі.

Головна сторінка (Генератор паролів)

Функція `home_view` відповідає за відображення головної сторінки додатку - Генератора. При зверненні до кореневої адреси (`/`) сервер повертає шаблон `home.html`. У шаблоні міститься графічний інтерфейс із повзунком для вибору довжини пароля (від 8 до 32 символів) та чекбоксами для вибору наборів символів (великі літери, цифри, спеціальні символи). Алгоритм генерації виконується миттєво на стороні клієнта за допомогою JavaScript, після чого користувач може натиснути кнопку "Зберегти", яка передає згенерований рядок через URL-параметр до форми збереження.

Модуль Авторизації (`register_view` та `login_view`)

Функція `register_view` забезпечує реєстрацію нових користувачів. При отриманні POST-запиту, функція виконує валідацію даних: перевіряє, чи співпадають введені паролі, та чи не зайнятий вже такий логін у базі даних. Якщо перевірка успішна, викликається метод `User.objects.create_user()`, який безпечно хешує пароль та створює новий запис. Функція `login_view` відображає екран входу в додаток. Основна функціональність полягає в автентифікації користувача. Після введення логіна та пароля, викликається метод `authenticate()`, який порівнює хеші. В разі успіху користувач авторизується за допомогою методу `login()` та перенаправляється на головний екран.

Збереження пароля (`save_password_view`)

Ця функція відповідає за додавання нових облікових даних до особистого Сховища користувача. Доступ до неї захищений декоратором `@login_required`, що гарантує виконання коду лише для авторизованих користувачів. При створенні запиту функція отримує згенерований пароль і підставляє його у HTML-форму. Після заповнення користувачем полів "Назва сервісу" та "Логін" і натискання кнопки, сервер отримує POST-запит, створює новий об'єкт `PasswordRecord` і записує його в базу даних, після чого автоматично переадресовує користувача до сторінки особистого Сховища.

Сховище паролів (`passwords_view`)

Клас-контролер `passwords_view` виступає як компонент, призначений для управління та відображення списку всіх збережених паролів користувача. Важливою особливістю є взаємодія з базою даних через Django ORM: виклик `PasswordRecord.objects.filter(user=request.user)` гарантує, що з бази будуть отримані лише ті записи, які належать поточній активній сесії. Отриманий список передається у шаблон `passwords.html`, де за допомогою циклу `{% for %}` формуються зручні картки для кожного збереженого ресурсу.

Перевірка надійності (`check_view`)

Клас `check_view` дозволяє користувачам перевіряти складність та стійкість до зламу вже існуючих паролів. Інтерфейс містить текстове поле та інтерактивну шкалу прогресу (Progress Bar). Алгоритм перевірки аналізує введений текст за кількома критеріями (наявність великих літер, цифр, спецсимволів, довжина) та динамічно змінює колір індикатора (від червоного "Слабкий" до зеленого "Надійний"). Дані, введені на цій сторінці, не передаються на сервер, що гарантує повну конфіденційність.

Особистий кабінет (`profile_view`)

Клас `profile_view` виступає як сторінка управління профілем користувача. Основним завданням цього контролера є надання можливості користувачеві переглядати статистику свого облікового запису (наприклад, загальну кількість збережених паролів) та редагувати інформацію..

У функції реалізована обробка двох різних POST-запитів:

1. **Оновлення профілю:** користувач має можливість змінити свій логін та електронну адресу. Сервер додатково перевіряє, чи не зайнятий новий логін іншим користувачем бази.
2. **Зміна пароля:** користувач може оновити свій майстер-пароль. Для того щоб після зміни пароля сесія користувача не обірвалася, використовується спеціальний метод `update_session_auth_hash`, який плавно оновлює дані

авторизації.

Панель адміністратора (custom_admin_view)

Для забезпечення керованості системою було розроблено кастомну панель адміністратора. На відміну від стандартних засобів Django, цей модуль інтегрований безпосередньо у загальний дизайн веб-додатку. Доступ до панелі суворо обмежений прапорцем `is_superuser`. Контролер отримує повний список зареєстрованих користувачів та дозволяє адміністратору виконувати адміністративні дії: оновлення контактних даних та скидання паролів у разі втрати доступу користувачем. Взаємодія з базою даних відбувається через метод `User.objects.get()`, а безпека зміни паролів гарантується методом `set_password()`, який автоматично перехешовує новий пароль.

РОЗДІЛ 4. КЕРІВНИЦТВО КОРИСТУВАЧА

Після запуску веб-сервера користувач переходить за відповідною URL-адресою та зустрічається з інтерфейсом веб-додатку PassGuard. Розглянемо графічний інтерфейс готового проекту та всі можливі варіанти взаємодії користувача із системою.

4.1. Екран авторизації

Після першого запуску застосунку, якщо користувач ще не має активної сесії, його зустрічає екран авторизації (Логіну). Тут він може увійти в систему, використовуючи свій логін та майстер-пароль. Інтерфейс виконаний у мінімалістичному світлому стилі із заокругленими формами для забезпечення максимального комфорту.

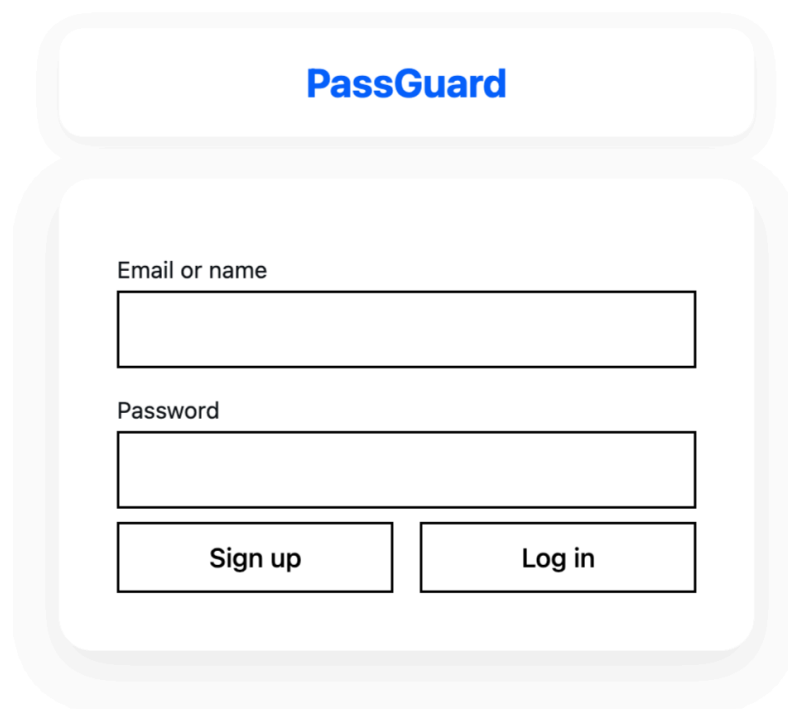
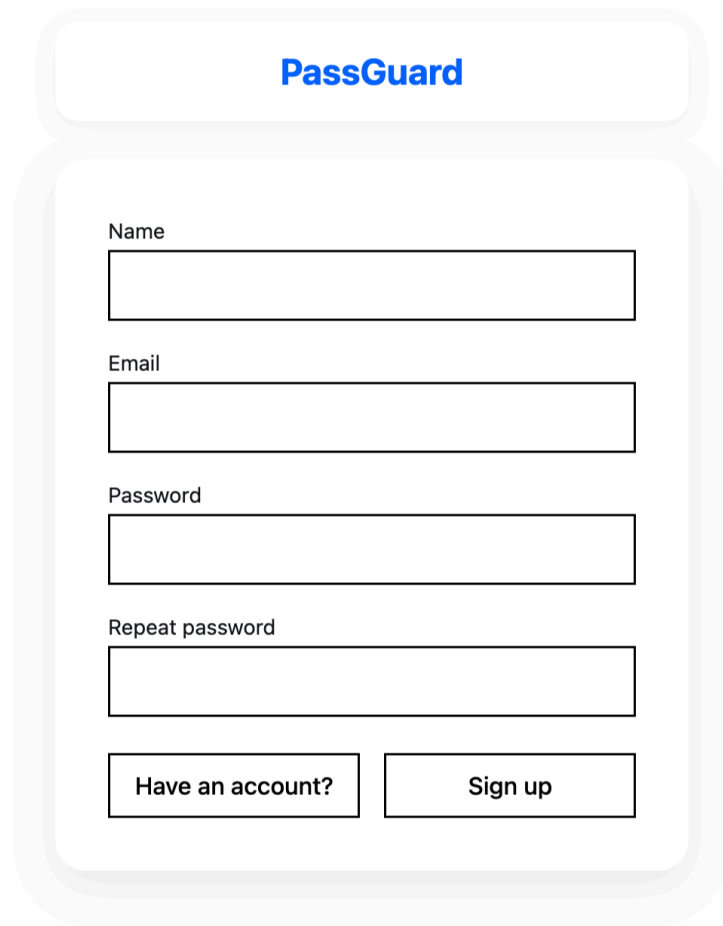
A mockup of the PassGuard login screen. It features a light gray background with rounded corners. At the top, the text "PassGuard" is displayed in a bold, blue font. Below this, there are two input fields: the first is labeled "Email or name" and the second is labeled "Password". Both labels are in a small, gray font. At the bottom, there are two buttons: "Sign up" and "Log in", both in a bold, black font. The entire form is enclosed in a light gray border with rounded corners.

Рисунок 4.1 – Екран авторизації

Якщо людина використовує застосунок вперше, їй потрібно створити обліковий запис, натиснувши на кнопку «Sign up» (Реєстрація). Тут на неї чекає форма, де необхідно ввести ім'я (логін), електронну адресу та двічі ввести пароль

для підтвердження. У разі введення некоректних даних (наприклад, паролі не співпадають), система виведе відповідне попередження.



The image shows a registration form titled "PassGuard" in blue text. The form is contained within a light gray rounded rectangle. It includes four input fields: "Name", "Email", "Password", and "Repeat password". At the bottom, there are two buttons: "Have an account?" and "Sign up".

PassGuard

Name

Email

Password

Repeat password

Have an account? Sign up

Рисунок 4.2 – Екран реєстрації користувача

4.2. Головний екран (Генератор паролів)

Після успішної авторизації або реєстрації, користувача зустрічає головний екран веб-додатку - Генератор надійних паролів. Інтерфейс розділено на дві основні частини: зліва знаходиться навігаційне бокове меню для швидкого переходу між розділами, а по центру - головна робоча зона.

У робочій зоні користувач бачить великий згенерований пароль. Під ним розташований блок «Налаштування складності», де можна взаємодіяти з параметрами генерації:

- За допомогою повзунка (слайдера) можна обрати бажану довжину пароля (від 8 до 32 символів).
- За допомогою перемикачів можна вмикати або вимикати використання великих літер, цифр та спеціальних символів.

При будь-якій зміні налаштувань, новий пароль генерується миттєво.

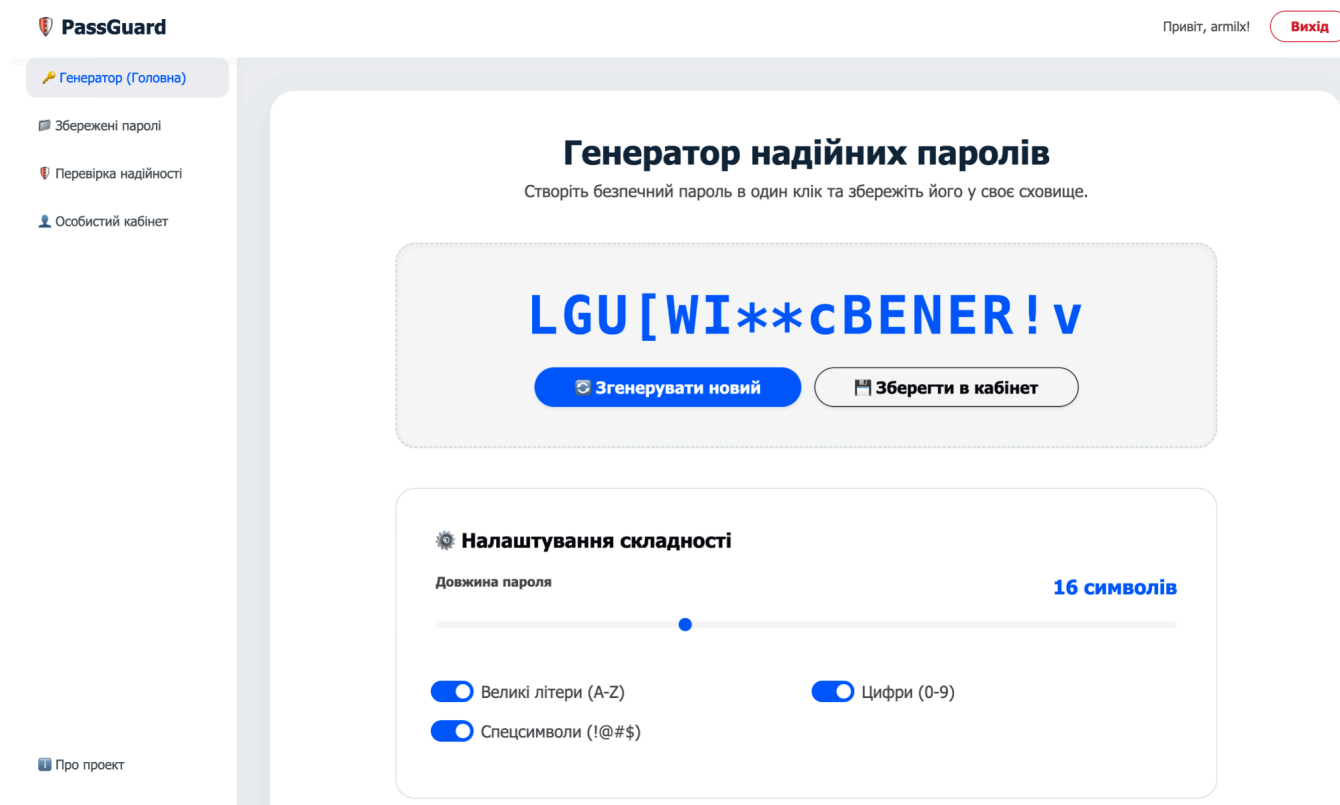


Рисунок 4.3 – Головний екран генератора паролів

4.3. Збереження пароля та Особисте сховище

Згенерувавши ідеальний пароль, користувач може натиснути кнопку «Зберегти в кабінет». Після цього відбувається перехід на екран збереження. Тут згенерований пароль вже автоматично вставлений у відповідне поле. Користувачеві залишається лише ввести назву сервісу (наприклад, "youtube") та свій логін чи електронну пошту для цього сервісу.

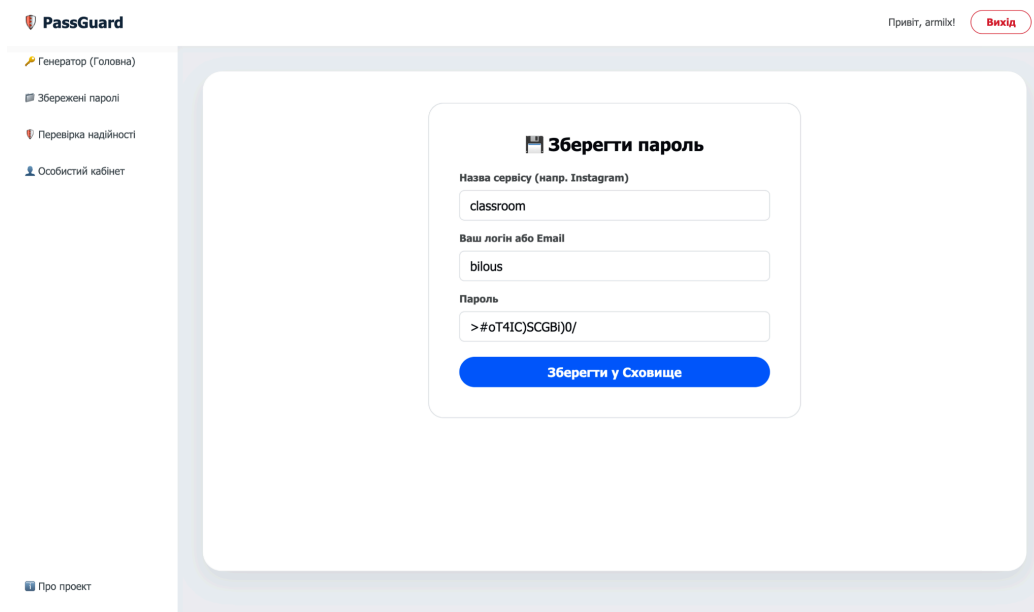


Рисунок 4.4 –Зберегти пароль

Після натискання кнопки збереження, користувача автоматично переміщує до розділу «Збережені паролі» (Особисте сховище). На цьому екрані всі збережені облікові дані відображаються у вигляді зручних карток. Кожна картка містить назву ресурсу, логін та сам пароль, який користувач може скопіювати для подальшого використання.

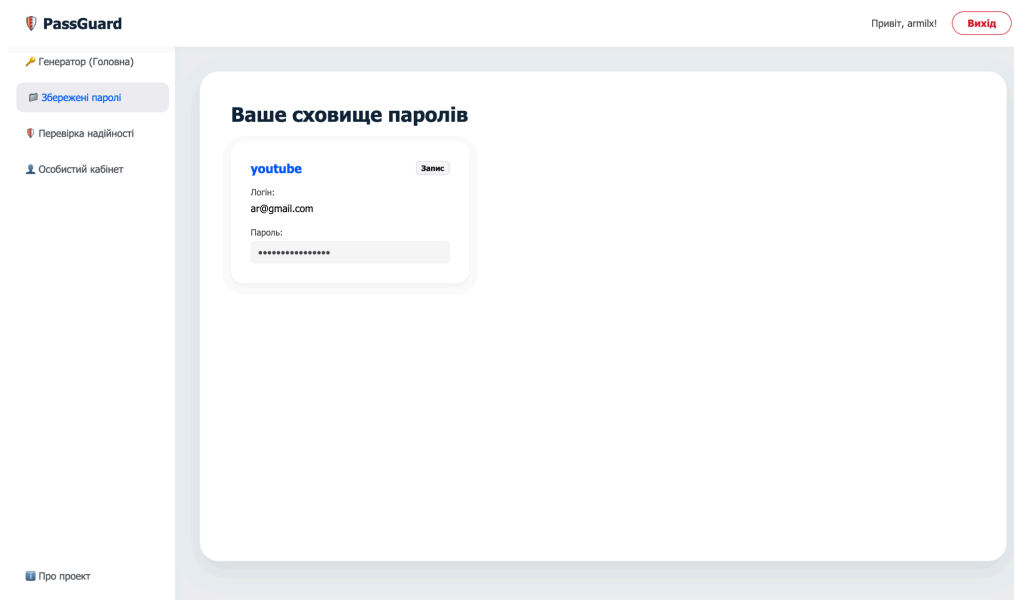


Рисунок 4.5 – Список збережених паролів користувача

4.4. Перевірка надійності пароля

Ще однією важливою функцією системи є інструмент перевірки існуючих паролів. Перейшовши до розділу «Перевірка надійності» через бокове меню, користувач може перевірити, наскільки безпечними є його старі паролі.

При введенні тексту у поле, інтерактивна шкала (Progress Bar) миттєво аналізує його складність та змінює колір: від червоного (Слабкий пароль) до жовтого (Нормальний) та зеленого (Надійний пароль).

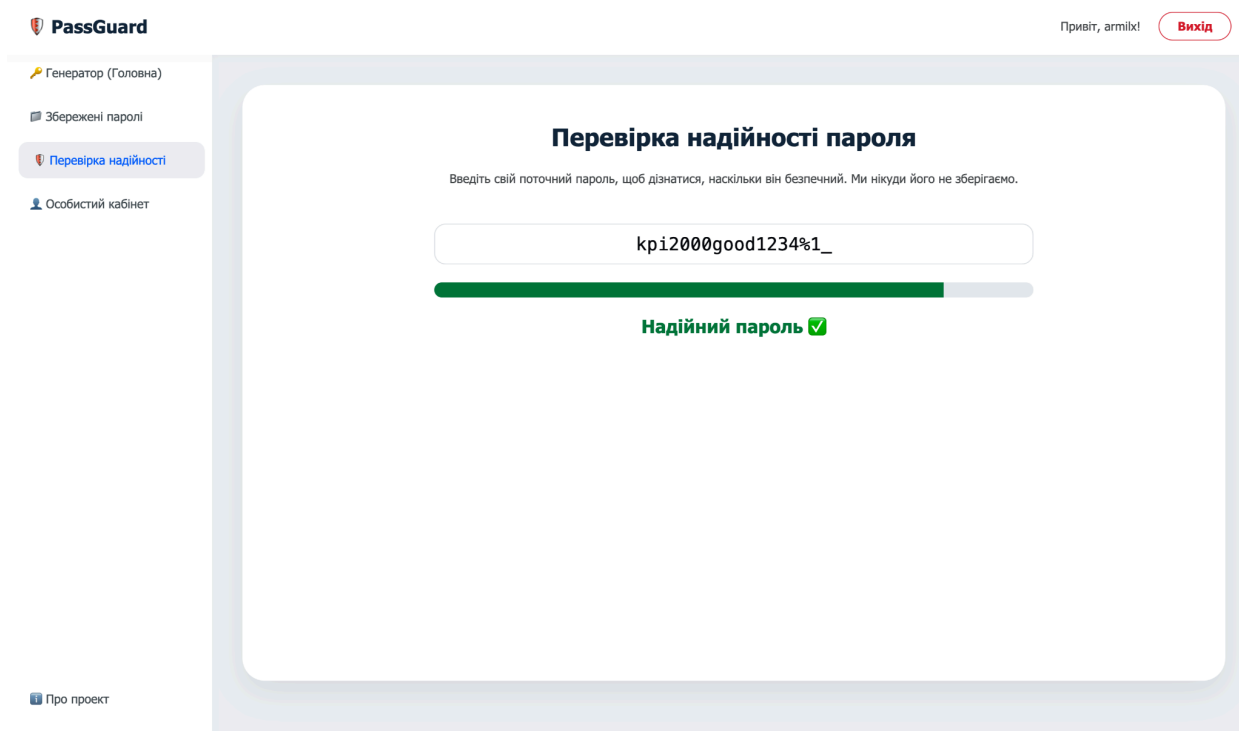


Рисунок 4.6 – Екран перевірки надійності пароля

4.5. Управління профілем (Особистий кабінет)

Останнім ключовим розділом є «Особистий кабінет». На цьому екрані користувач може керувати своїм обліковим записом.

У лівій частині екрану відображається інформація про поточного користувача, кнопка виходу з системи (Logout) та великий лічильник, який показує загальну кількість збережених паролів у Сховищі.

У правій частині екрану розташовані дві форми для редагування профілю:

- Зміна особистих даних: дозволяє оновити свій логін та електронну пошту.

- Зміна майстер-пароля: дозволяє задати новий пароль для входу в систему PassGuard.

Після успішної зміни даних система виводить зелене повідомлення про успішне збереження налаштувань.

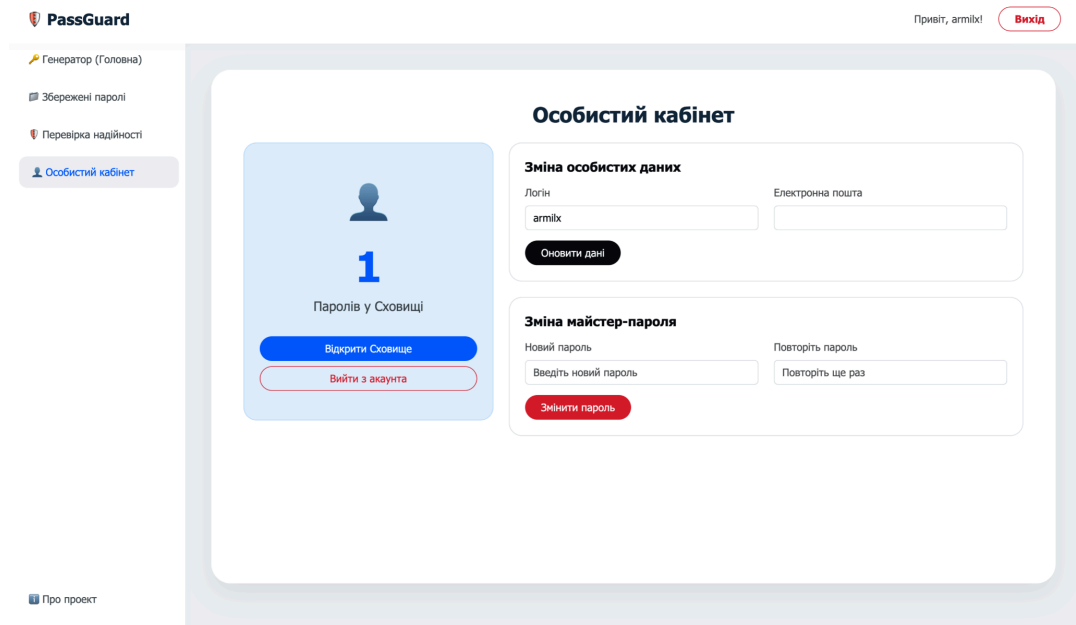


Рисунок 4.7 – Екран особистого кабінету та налаштувань профілю

4.6 Панель адміністратора

Якщо в систему входить користувач із правами адміністратора, у боковому меню з'являється додатковий розділ «Панель Адміна». На цьому екрані відображається таблиця з усіма користувачами системи. Адміністратор має можливість в один клік оновити електронну пошту будь-якого користувача або задати йому новий пароль через відповідні форми у таблиці.

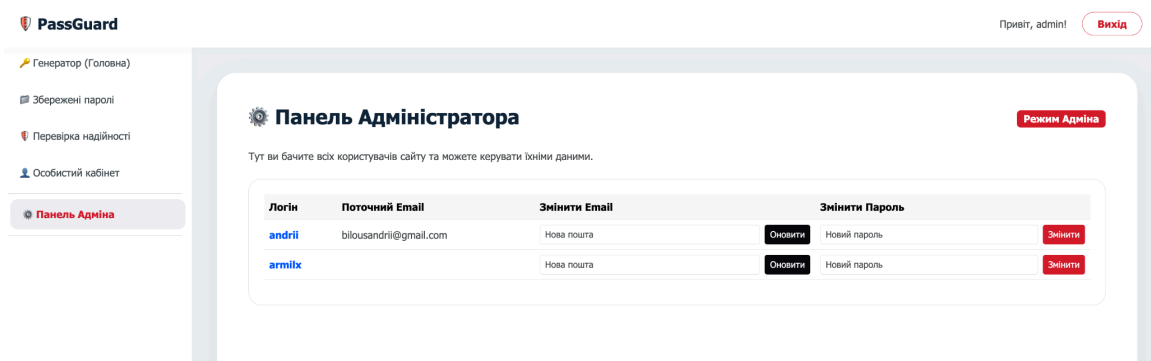


Рисунок 4.8 – Екран панелі адміністратора

ВИСНОВКИ

У ході виконання цієї курсової роботи вдалося розробити та успішно впровадити безпечний веб-додаток - менеджер паролів «PassGuard», який представляє значну практичну цінність у сфері інформаційної безпеки. Застосування сучасної мови програмування Python та потужного веб-фреймворку Django, а також інструментів фронтенд-розробки (HTML5, CSS3, JavaScript та Bootstrap 5), дозволило реалізувати інтуїтивний, адаптивний та захищений інтерфейс.

Важливим аспектом є успішне вирішення завдань, пов'язаних із генерацією криптографічно стійких паролів, перевіркою їхньої надійності в реальному часі та безпечним збереженням у реляційній базі даних. Dodatok також повноцінно підтримує ізоляцію користувацьких сесій, систему автентифікації та управління особистим профілем.

Проект є надзвичайно актуальним у сучасному цифровому світі, де кількість облікових записів кожної людини стрімко зростає, а використання однакових чи слабких паролів є головною причиною витоку конфіденційних даних. Реалізація додатку відображає практичну необхідність у менеджерах паролів, надаючи зручний інструмент для підтримання цифрової гігієни та захисту ідентичності в мережі.

Ось деякі конкретні досягнення, яких вдалося досягти в рамках цього проекту:

- **Інтуїтивний та ефективний інтерфейс:** додаток має простий і зрозумілий дизайн, який дозволяє користувачам легко генерувати, зберігати та переглядати свої паролі як з комп'ютерів, так і з мобільних пристроїв.
- **Високий рівень безпеки:** використання вбудованих механізмів захисту Django (від CSRF, XSS, SQL-ін'єкцій) та безпечне хешування майстер-паролів гарантують цілісність та конфіденційність даних користувачів.

- **Розширені функції:** наявність інтерактивного аналізатора стійкості паролів та гнучкого генератора роблять процес управління обліковими даними максимально зручним.

У цілому, розробка цього веб-додатку була успішною. Додаток є функціональним, надійним і повністю готовим до використання інструментом для організації та захисту конфіденційної інформації.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

Python 3 Official Documentation [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.python.org/3/>

Django Documentation [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.djangoproject.com/>

Bootstrap 5.3 Documentation [Електронний ресурс] – Режим доступу до ресурсу: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

MDN Web Docs - JavaScript & HTML [Електронний ресурс] – Режим доступу до ресурсу: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Розробка безпечних веб-застосунків (OWASP Top 10) [Електронний ресурс] – Режим доступу до ресурсу: <https://owasp.org/www-project-top-ten/>

Django Authentication System [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.djangoproject.com/en/stable/topics/auth>

ДОДАТОК 1

Діаграми

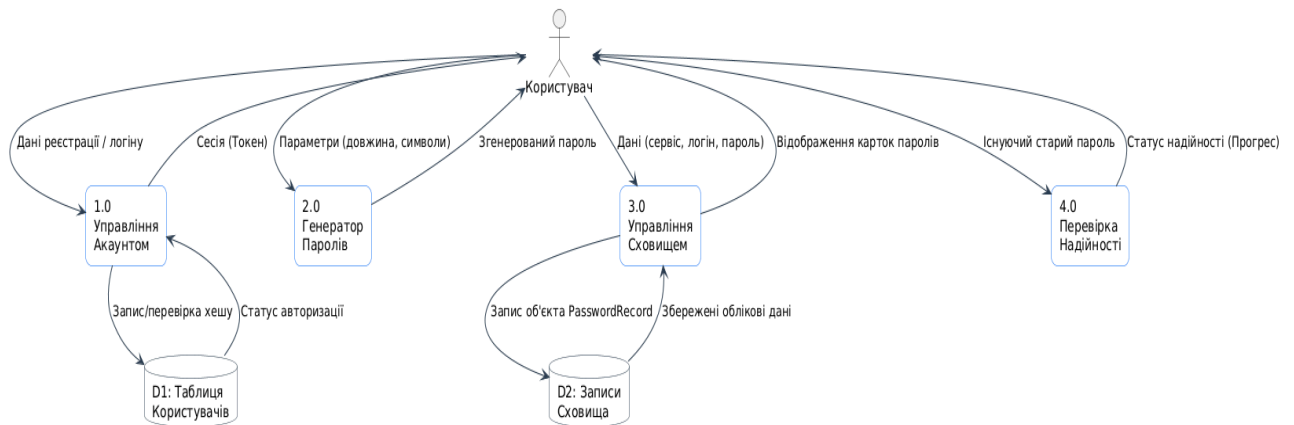
НТУУ «КПІ ім. І. Сікорського» ІАТЕ ТВ-43

Листів 2

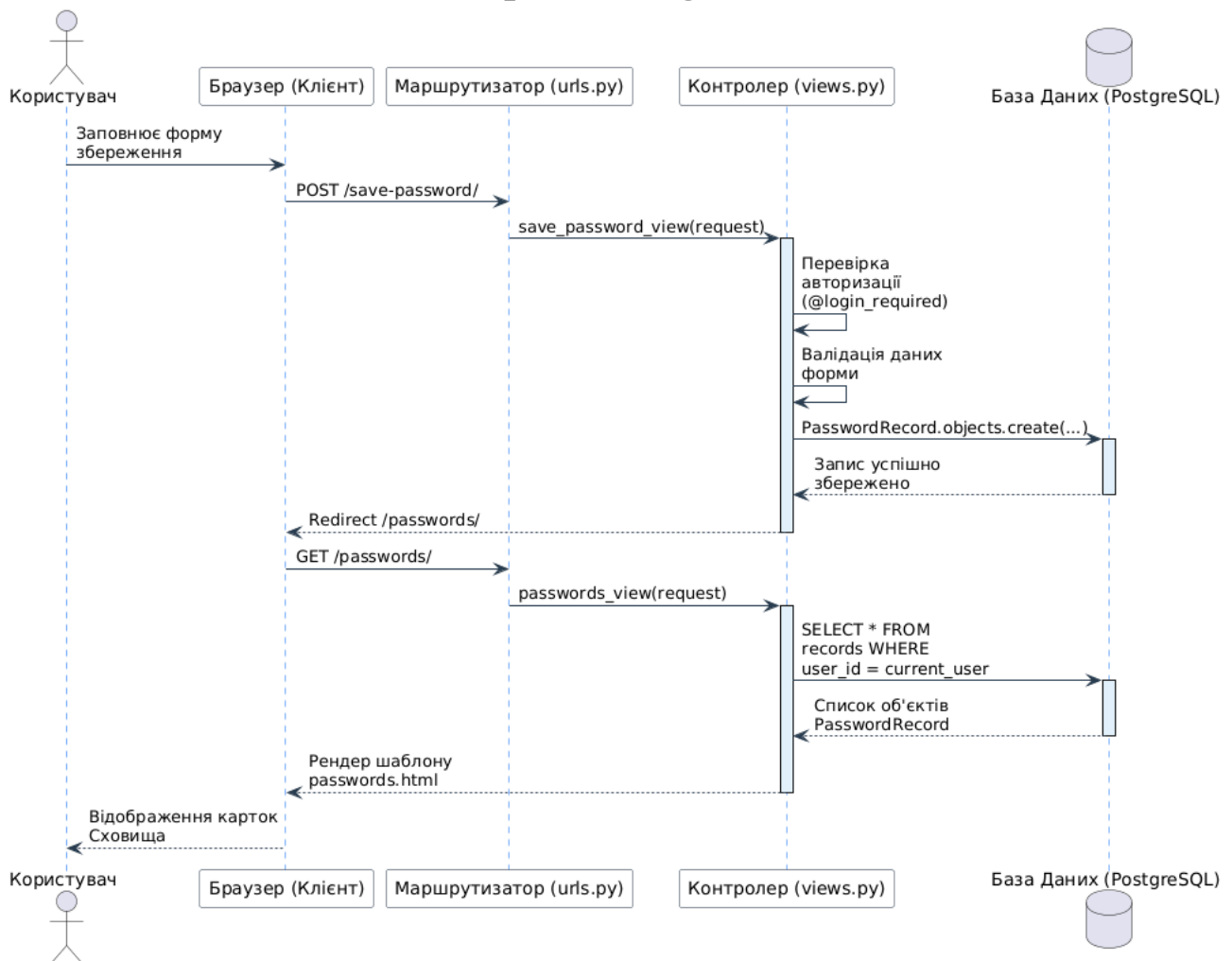
Use case diagram



Dfd diagram



Sequence diagram



ДОДАТОК 2

Опис програми

НТУУ «КПІ ім. І. Сікорського» ІАТЕ ТВ-43

Листів 2

В ході виконання даної курсової роботи було створено веб-додаток, який містить необхідний набір функцій для безпечного управління паролями. Програма була реалізована за допомогою мови програмування Python, фреймворку Django та базових веб-технологій (HTML5, CSS3, JavaScript).

Цей курсовий проект мав на меті розробити сучасний менеджер паролів, який буде відповідати актуальним потребам користувачів у сфері кібербезпеки. Додаток був розроблений за архітектурою MVT (Model-View-Template) і використовує реляційну базу даних (SQLite/PostgreSQL) для безпечного зберігання зашифрованої інформації.

Основні завдання додатку включали в себе:

- **Зручність використання:** Додаток є простим у використанні, має інтуїтивно зрозумілий та адаптивний інтерфейс, що коректно відображається на різних пристроях.
- **Функціональність:** Додаток забезпечує широкий спектр можливостей для генерації, аналізу та надійного збереження облікових даних.
- **Надійність та безпека:** Додаток стійкий до поширених веб-вразливостей, надійно захищає дані авторизованих користувачів і працює без помилок.

Актуальність даної теми зумовлена стрімким зростанням кількості цифрових сервісів та пов'язаних із цим кіберзагроз. Менеджери паролів є незамінним інструментом для людей, які прагнуть захистити свою конфіденційну інформацію та уникнути використання однакових паролів на різних сайтах. Розробка сучасного веб-додатку для управління паролями є важливим вкладом у підвищення рівня цифрової гігієни суспільства.

Вся серверна частина програми була реалізована мовою програмування Python. Розробка, написання коду та тестування проводилися в інтегрованому середовищі розробки Visual Studio Code.

Конкретні функціональні можливості додатку:

- **Генерація надійних паролів:** Користувачі можуть створювати стійкі до зламу паролі, гнучко налаштовуючи їхню довжину (від 8 до 32 символів) та набір символів (великі літери, цифри, спецсимволи).
- **Збереження облікових даних:** Додаток дозволяє користувачам зберігати згенеровані паролі у прив'язці до конкретних сервісів у своєму ізольованому особистому Сховищі.
- **Перевірка надійності:** Наявність вбудованого інструменту для аналізу складності вже існуючих паролів користувача в режимі реального часу за допомогою інтерактивної шкали.
- **Управління профілем:** Забезпечено можливість безпечного оновлення особистих даних (логін, пошта) та зміни майстер-пароля.

Очікувані результати:

Після завершення розробки додаток повністю відповідає наступним вимогам:

- Додаток простий у використанні та має сучасний візуально привабливий інтерфейс.
- Додаток забезпечує повноцінний набір інструментів для створення та управління обліковими даними.
- Додаток працює стабільно, а дані користувачів надійно захищені від несанкціонованого доступу.

Розробка веб-додатку менеджера паролів є важливим завданням, яке має величезний потенціал для покращення безпеки сучасних користувачів у мережі Інтернет. Додаток, розроблений у рамках цього курсового проекту, повністю задовольняє ці потреби та забезпечує надійний рівень захисту інформації.

ДОДАТОК 3

Код програми

НТУУ «КПІ ім. І. Сікорського» ІАТЕ ТВ-43

Листів 15

Київ – 2026

asgi.py

```
import os

from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')

application = get_asgi_application()
```

settings.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-$mdyx*9n8#onbxcr1155@__pea70iqc3hm%%ia3la035fdxu55'

DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'manager',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'core.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'core.wsgi.application'

DATABASES = {
```

```

'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': BASE_DIR / 'db.sqlite3',
}
}

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

STATIC_URL = 'static/'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

```

urls.py

```

from django.contrib import admin
from django.urls import path
from manager import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home_view, name='home'),
    path('login/', views.login_view, name='login'),
    path('register/', views.register_view, name='register'),
    path('logout/', views.logout_view, name='logout'),
    path('save-password/', views.save_password_view, name='save_password'),
    path('passwords/', views.passwords_view, name='passwords'),
    path('check/', views.check_view, name='check'),
    path('profile/', views.profile_view, name='profile'),
    path('about/', views.about_view, name='about'),
    path('admin-panel/', views.custom_admin_view, name='custom_admin'),
]

```

wsgi.py

```

import os

from django.core.wsgi import get_wsgi_application

```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
```

```
application = get_wsgi_application()
```

about.html

```
{% extends 'manager/base.html' %}
{% block content %}
<div class="p-5">
  <h2 class="fw-bold mb-4" style="color: #2c3e50;">Про проект PassGuard 
```

base.html

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PassGuard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { background-color: #eef2f5; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }

    .navbar { background-color: #ffffff; box-shadow: 0 2px 10px rgba(0,0,0,0.03); padding: 15px 20px; border-bottom:
    none; }
    .navbar-brand { font-weight: 700; color: #2c3e50; font-size: 24px; }

    .sidebar { height: calc(100vh - 70px); background-color: #ffffff; padding-top: 30px; position: relative; box-shadow:
    2px 0 10px rgba(0,0,0,0.02);}
```

```

.sidebar a { color: #5a6268; text-decoration: none; padding: 12px 20px; display: block; border-radius: 12px; margin: 0 15px 10px 15px; font-weight: 500; transition: all 0.3s ease; }
.sidebar a:hover, .sidebar a.active { background-color: #eef2f5; color: #0d6efd; transform: translateX(5px); }
.about-btn { position: absolute; bottom: 30px; width: calc(100% - 30px); }

```

```

.main-content { padding: 40px; }
.content-card { background: white; border-radius: 30px; padding: 50px; box-shadow: 0 15px 40px rgba(0,0,0,0.08); min-height: 80vh; }

```

```

</style>
</head>
<body>
  <nav class="navbar navbar-expand-lg sticky-top">
    <div class="container-fluid">
      <a class="navbar-brand" href="/"><img alt="PassGuard logo" data-bbox="395 228 415 245"/> PassGuard</a>
      <div class="d-flex align-items-center">
        {% if request.user.is_authenticated %}
          <span class="me-4 fw-medium text-muted">Привіт, {{ request.user.username }}!</span>
          <a href="/logout/" class="btn btn-outline-danger rounded-pill px-4 fw-bold">Вихід</a>
        {% else %}
          <a href="/login/" class="btn btn-primary rounded-pill px-4 fw-bold">Увійти</a>
        {% endif %}
      </div>
    </div>
  </nav>

  <div class="container-fluid">
    <div class="row">
      <div class="col-md-2 p-0 sidebar d-none d-md-block">
        <a href="/" class="{% if request.path == '/' %}active{% endif %}"><img alt="Key icon" data-bbox="635 435 655 450"/> Генератор (Головна)</a>
        <a href="/passwords/" class="{% if request.path == '/passwords/' %}active{% endif %}"><img alt="Folder icon" data-bbox="785 448 805 463"/> Збережені
        паролі</a>
        <a href="/check/" class="{% if request.path == '/check/' %}active{% endif %}"><img alt="Shield icon" data-bbox="725 475 745 490"/> Перевірка
        надійності</a>
        <a href="/profile/" class="{% if request.path == '/profile/' %}active{% endif %}"><img alt="User icon" data-bbox="735 502 755 517"/> Особистий кабінет</a>

        <a href="/about/" class="about-btn {% if request.path == '/about/' %}active{% endif %}"><img alt="Info icon" data-bbox="785 529 805 544"/> Про проект</a>
      </div>

      <div class="col-md-10 main-content">
        <div class="content-card">
          {% block content %}
          {% endblock %}
        </div>
      </div>
    </div>
  </div>
</body>
</html>

```

check.html

```

{% extends 'manager/base.html' %}
{% block content %}
<div class="row justify-content-center">
  <div class="col-md-8 text-center">
    <h2 class="fw-bold mb-4" style="color: #2c3e50;">Перевірка надійності пароля</h2>
    <p class="text-muted mb-5">Введіть свій поточний пароль, щоб дізнатися, наскільки він безпечний. Ми нікуди його не зберігаємо.</p>

    <input type="text" id="pwd-input" class="form-control form-control-lg text-center font-monospace mb-4"
    placeholder="Введіть пароль..." style="border-radius: 15px; font-size: 24px;">

    <div class="progress" style="height: 20px; border-radius: 10px;">

```

```

        <div id="strength-bar" class="progress-bar" role="progressbar" style="width: 0%; transition: width 0.4s
ease;"></div>
    </div>
    <h4 id="strength-text" class="mt-4 fw-bold text-muted">Введіть пароль для перевірки</h4>
</div>
</div>

<script>
    document.getElementById('pwd-input').addEventListener('input', function(e) {
        let val = e.target.value;
        let score = 0;

        if (val.length > 0) score += 10;
        if (val.length >= 8) score += 20;
        if (val.length >= 12) score += 20;
        if (/[A-Z]/.test(val)) score += 15;
        if (/[0-9]/.test(val)) score += 15;
        if (/^[A-Za-z0-9]/.test(val)) score += 20;

        let bar = document.getElementById('strength-bar');
        let text = document.getElementById('strength-text');

        bar.style.width = score + '%';

        if (val.length === 0) {
            bar.style.width = '0%'; text.innerText = 'Введіть пароль для перевірки'; text.className = 'mt-4 fw-bold
text-muted';
        } else if (score < 40) {
            bar.className = 'progress-bar bg-danger'; text.innerText = 'Слабкий пароль 🚫'; text.className = 'mt-4 fw-bold
text-danger';
        } else if (score < 80) {
            bar.className = 'progress-bar bg-warning'; text.innerText = 'Нормальний пароль ⚠️'; text.className = 'mt-4
fw-bold text-warning';
        } else {
            bar.className = 'progress-bar bg-success'; text.innerText = 'Надійний пароль ✅'; text.className = 'mt-4
fw-bold text-success';
        }
    });
</script>
{% endblock %}

```

custom_admin.html

```

{% extends 'manager/base.html' %}
{% block content %}
<div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="fw-bold" style="color: #2c3e50;">⚙️ Панель Адміністратора</h2>
    <span class="badge bg-danger fs-6">Режим Адміна</span>
</div>
<p class="text-muted mb-4">Тут ви бачите всіх користувачів сайту та можете керувати їхніми даними.</p>

{% if success_msg %}
<div class="alert alert-success fw-bold">{{ success_msg }}</div>
{% endif %}

<div class="table-responsive bg-white border p-4" style="border-radius: 20px;">
    <table class="table table-hover align-middle">
        <thead class="table-light">
            <tr>
                <th>Логін</th>
                <th>Поточний Email</th>
                <th>Змінити Email</th>
                <th>Змінити Пароль</th>
            </tr>
        </thead>
    </table>
</div>

```

```

</tr>
</thead>
<tbody>
  {% for u in users %}
    <tr>
      <td class="fw-bold text-primary">{{ u.username }}</td>
      <td class="text-muted">{{ u.email }}</td>

      <td>
        <form method="POST" class="d-flex gap-2">
          {% csrf_token %}
          <input type="hidden" name="action" value="update_email">
          <input type="hidden" name="user_id" value="{{ u.id }}">
          <input type="email" name="new_email" class="form-control form-control-sm" placeholder="Нова
пошта" required>
          <button type="submit" class="btn btn-sm btn-dark">Оновити</button>
        </form>
      </td>

      <td>
        <form method="POST" class="d-flex gap-2">
          {% csrf_token %}
          <input type="hidden" name="action" value="update_password">
          <input type="hidden" name="user_id" value="{{ u.id }}">
          <input type="password" name="new_password" class="form-control form-control-sm"
placeholder="Новий пароль" required>
          <button type="submit" class="btn btn-sm btn-danger">Змінити</button>
        </form>
      </td>
    </tr>
    {% empty %}
    <tr>
      <td colspan="4" class="text-center text-muted py-4">Поки що немає зареєстрованих користувачів крім
вас.</td>
    </tr>
  {% endfor %}
</tbody>
</table>
</div>
{% endblock %}

```

home.html

```

{% extends 'manager/base.html' %}

{% block content %}
<div class="row justify-content-center">
  <div class="col-lg-10">
    <div class="text-center mb-5">
      <h2 class="fw-bold" style="color: #2c3e50; font-size: 2.5rem;">Генератор надійних паролів</h2>
      <p class="text-muted fs-5">Створіть безпечний пароль в один клік та збережіть його у своє сховище.</p>
    </div>

    <div class="p-5 bg-light text-center mb-5" style="border-radius: 24px; box-shadow: inset 0 2px 10px
rgba(0,0,0,0.03); border: 2px dashed #dee2e6;">
      <h1 id="password-display" class="display-3 fw-bold font-monospace mb-4" style="color: #0d6efd;
letter-spacing: 4px; word-break: break-all;"></h1>

      <div class="d-flex justify-content-center gap-3 mt-4">
        <button id="btn-generate" class="btn btn-primary btn-lg rounded-pill px-5 shadow-sm fw-bold">↺
Згенерувати новий</button>
        <a id="btn-save" href="#" class="btn btn-outline-dark btn-lg rounded-pill px-5 shadow-sm fw-bold">💾
Зберегти в кабінет</a>
      </div>
    </div>
  </div>
</div>

```

```

</div>

<div class="p-5 border" style="border-radius: 24px; background-color: #ffffff;">
  <h4 class="fw-bold mb-4 text-dark">⚙️ Налаштування складності</h4>

  <div class="mb-5">
    <label class="form-label fw-bold d-flex justify-content-between text-muted mb-3">
      <span>Довжина пароля</span>
      <span id="length-val" class="text-primary fs-4 fw-bold">16 символів</span>
    </label>
    <input type="range" id="pwd-length" class="form-range" min="8" max="32" value="16">
  </div>

  <div class="row fs-5 text-muted">
    <div class="col-md-6 mb-3">
      <div class="form-check form-switch form-switch-lg">
        <input class="form-check-input" type="checkbox" id="chk-upper" checked style="transform: scale(1.3); margin-right: 15px;">
        <label class="form-check-label fw-medium">Великі літери (A-Z)</label>
      </div>
    </div>
    <div class="col-md-6 mb-3">
      <div class="form-check form-switch form-switch-lg">
        <input class="form-check-input" type="checkbox" id="chk-numbers" checked style="transform: scale(1.3); margin-right: 15px;">
        <label class="form-check-label fw-medium">Цифри (0-9)</label>
      </div>
    </div>
    <div class="col-md-6 mb-3">
      <div class="form-check form-switch form-switch-lg">
        <input class="form-check-input" type="checkbox" id="chk-symbols" checked style="transform: scale(1.3); margin-right: 15px;">
        <label class="form-check-label fw-medium">Спецсимволи (!@#%$^&*()_+~`|} {[]:;?><./-=">
      </div>
    </div>
  </div>
</div>

<script>
  document.addEventListener("DOMContentLoaded", function() {
    const display = document.getElementById('password-display');
    const lengthSlider = document.getElementById('pwd-length');
    const lengthVal = document.getElementById('length-val');
    const chkUpper = document.getElementById('chk-upper');
    const chkNumbers = document.getElementById('chk-numbers');
    const chkSymbols = document.getElementById('chk-symbols');
    const btnGenerate = document.getElementById('btn-generate');
    const btnSave = document.getElementById('btn-save');

    function generatePassword() {
      let charset = "abcdefghijklmnopqrstuvwxyz";
      if (chkUpper.checked) charset += "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
      if (chkNumbers.checked) charset += "0123456789";
      if (chkSymbols.checked) charset += "!@#%$^&*()_+~`|} {[]:;?><./-=";

      let length = lengthSlider.value;
      let retVal = "";
      for (let i = 0, n = charset.length; i < length; ++i) {
        retVal += charset.charAt(Math.floor(Math.random() * n));
      }
      display.innerText = retVal;
      btnSave.href = "/save-password/?pwd=" + encodeURIComponent(retVal);
    }
  });

```

```

lengthSlider.addEventListener('input', function() {
    lengthVal.innerText = this.value + " символів";
    generatePassword();
});
btnGenerate.addEventListener('click', generatePassword);
chkUpper.addEventListener('change', generatePassword);
chkNumbers.addEventListener('change', generatePassword);
chkSymbols.addEventListener('change', generatePassword);
generatePassword();
});
</script>
{% endblock %}

```

login.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Вхід - PassGuard</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        body { background-color: #ffffff; display: flex; justify-content: center; align-items: center; min-height: 100vh;
margin: 0; }
        .auth-container { width: 100%; max-width: 450px; padding: 15px; }
        .form-box { border: none; border-radius: 20px; padding: 35px; background: white; box-shadow: 0 10px 30px
rgba(0,0,0,0.08); }
        .logo-box { border: none; border-radius: 15px; padding: 15px; text-align: center; margin-bottom: 25px; font-weight:
bold; font-size: 24px; background: white; box-shadow: 0 4px 15px rgba(0,0,0,0.05); color: #0d6efd; }
        .form-control { border-radius: 10px; border: 1px solid #ddd; padding: 10px 15px; }
        .btn-custom { border-radius: 10px; border: 1px solid #0d6efd; color: #0d6efd; font-weight: 500; }
        .btn-custom:hover { background: #0d6efd; color: white; }
        .form-control { border: 1.5px solid #000; border-radius: 0; margin-bottom: 5px; box-shadow: none; }
        .form-control:focus { border-color: #000; box-shadow: none; }
        .btn-custom { border: 1.5px solid #000; background: white; color: black; border-radius: 0; padding: 8px; }
        .btn-custom:hover { background: #f0f0f0; color: black; }
        label { margin-bottom: 2px; font-size: 14px; margin-top: 10px; }
        .forgot-link { font-size: 12px; color: black; text-decoration: none; display: block; text-align: right; margin-bottom:
25px; }
        .forgot-link:hover { text-decoration: underline; }
    </style>
</head>
<body>
    <div class="auth-container">
        <div class="logo-box">PassGuard</div>
        <div class="form-box">
            <form method="POST">
                {% csrf_token %}
                {% if error %}
                <div class="alert alert-danger p-2 text-center" style="font-size: 14px;">{{ error }}</div>
                {% endif %}

                <label>Email or name</label>
                <input type="text" name="username" class="form-control" required>

                <label>Password</label>
                <input type="password" name="password" class="form-control" required>

                <div class="d-flex justify-content-between mt-2 gap-3">
                    <a href="/register/" class="btn btn-custom w-50 text-center">Sign up</a>
                    <button type="submit" class="btn btn-custom w-50">Log in</button>
                </div>
            </form>
        </div>
    </div>

```

```
</div>
</body>
</html>
```

passwords.html

```
{% extends 'manager/base.html' %}

{% block content %}
<h2 class="fw-bold mb-4" style="color: #2c3e50;">Ваше сховище паролів</h2>

<div class="row">
  {% for record in records %}
  <div class="col-md-6 col-lg-4 mb-4">
    <div class="card border-0 p-3" style="border-radius: 20px; box-shadow: 0 5px 15px rgba(0,0,0,0.05);">
      <div class="card-body">
        <div class="d-flex justify-content-between align-items-center mb-3">
          <h5 class="card-title fw-bold mb-0 text-primary">{{ record.service_name }}</h5>
          <span class="badge bg-light text-dark border">Запис</span>
        </div>
        <p class="text-muted mb-1 small">Логін:</p>
        <p class="fw-medium text-dark">{{ record.login }}</p>

        <p class="text-muted mb-1 small">Пароль:</p>
        <div class="input-group">
          <input type="password" class="form-control bg-light border-0 text-muted" value="{{
record.encrypted_password }}" readonly>
        </div>
      </div>
    </div>
  </div>
  {% empty %}
  <div class="col-12 text-center p-5">
    <h4 class="text-muted">У вас ще немає збережених паролів.</h4>
    <a href="/" class="btn btn-primary mt-3 rounded-pill px-4">Згенерувати перший пароль</a>
  </div>
  {% endfor %}
</div>
{% endblock %}
```

profile.html

```
{% extends 'manager/base.html' %}
{% block content %}
<h2 class="fw-bold mb-4 text-center" style="color: #2c3e50;">Особистий кабінет</h2>
{% if success_msg %}
<div class="alert alert-success text-center fw-bold">{{ success_msg }}</div>
{% endif %}
{% if error_msg %}
<div class="alert alert-danger text-center fw-bold">{{ error_msg }}</div>
{% endif %}

<div class="row">
  <div class="col-md-4 mb-4">
    <div class="p-4 border h-100 d-flex flex-column justify-content-center align-items-center text-center"
style="border-radius: 20px; background-color: #e3f2fd; border-color: #bbdefb !important;">
      <div style="font-size: 60px;">👤</div>
      <h1 class="display-3 fw-bold text-primary mb-0 mt-3">{{ saved_count }}</h1>
      <p class="text-muted fs-5 fw-medium mt-2">Паролів у Сховищі</p>
      <a href="/passwords/" class="btn btn-primary rounded-pill px-4 mt-3 w-100">Відкрити Сховище</a>
      <a href="/logout/" class="btn btn-outline-danger w-100 rounded-pill mt-2">Вийти з акаунта</a>
    </div>
  </div>
</div>
```

```

</div>

<div class="col-md-8">
  <div class="p-4 border mb-4" style="border-radius: 20px; background-color: #ffffff;">
    <h5 class="fw-bold text-dark mb-3">Зміна особистих даних</h5>
    <form method="POST">
      {% csrf_token %}
      <input type="hidden" name="update_profile" value="1">
      <div class="row">
        <div class="col-md-6 mb-3">
          <label class="form-label text-muted">Логін</label>
          <input type="text" name="username" class="form-control" value="{{ request.user.username }}"
required>
        </div>
        <div class="col-md-6 mb-3">
          <label class="form-label text-muted">Електронна пошта</label>
          <input type="email" name="email" class="form-control" value="{{ request.user.email }}" required>
        </div>
      </div>
      <button type="submit" class="btn btn-dark rounded-pill px-4">Оновити дані</button>
    </form>
  </div>

  <div class="p-4 border" style="border-radius: 20px; background-color: #ffffff;">
    <h5 class="fw-bold text-dark mb-3">Зміна майстер-пароля</h5>
    <form method="POST">
      {% csrf_token %}
      <input type="hidden" name="change_password" value="1">
      <div class="row">
        <div class="col-md-6 mb-3">
          <label class="form-label text-muted">Новий пароль</label>
          <input type="password" name="new_password" class="form-control" placeholder="Введіть новий
пароль" required>
        </div>
        <div class="col-md-6 mb-3">
          <label class="form-label text-muted">Повторіть пароль</label>
          <input type="password" name="confirm_password" class="form-control" placeholder="Повторіть ще
раз" required>
        </div>
      </div>
      <button type="submit" class="btn btn-danger rounded-pill px-4">Змінити пароль</button>
    </form>
  </div>
</div>
{% endblock %}

```

register.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ресурсація - PassGuard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { background-color: #ffffff; display: flex; justify-content: center; align-items: center; min-height: 100vh;
margin: 0; }
    .auth-container { width: 100%; max-width: 450px; padding: 15px; }
    .form-box { border: none; border-radius: 20px; padding: 35px; background: white; box-shadow: 0 10px 30px
rgba(0,0,0,0.08); }

```

```

        .logo-box { border: none; border-radius: 15px; padding: 15px; text-align: center; margin-bottom: 25px; font-weight:
bold; font-size: 24px; background: white; box-shadow: 0 4px 15px rgba(0,0,0,0.05); color: #0d6efd;}
        .form-control { border-radius: 10px; border: 1px solid #ddd; padding: 10px 15px; }
        .btn-custom { border-radius: 10px; border: 1px solid #0d6efd; color: #0d6efd; font-weight: 500; }
        .btn-custom:hover { background: #0d6efd; color: white; }
        .form-control { border: 1.5px solid #000; border-radius: 0; margin-bottom: 15px; box-shadow: none; }
        .form-control:focus { border-color: #000; box-shadow: none; }
        .btn-custom { border: 1.5px solid #000; background: white; color: black; border-radius: 0; padding: 8px; }
        .btn-custom:hover { background: #f0f0f0; color: black; }
        label { margin-bottom: 2px; font-size: 14px; }
    </style>
</head>
<body>
    <div class="auth-container">
        <div class="logo-box">PassGuard</div>
        <div class="form-box">
            <form method="POST">
                {% csrf_token %}
                {% if error %}
                <div class="alert alert-danger p-2 text-center" style="font-size: 14px;">{{ error }}</div>
                {% endif %}

                <label>Name</label>
                <input type="text" name="username" class="form-control" required>

                <label>Email</label>
                <input type="email" name="email" class="form-control" required>

                <label>Password</label>
                <input type="password" name="password" class="form-control" required>

                <label>Repeat password</label>
                <input type="password" name="password_confirm" class="form-control" required>

                <div class="d-flex justify-content-between mt-4 gap-3">
                    <a href="/login/" class="btn btn-custom w-50">Have an account?</a>
                    <button type="submit" class="btn btn-custom w-50">Sign up</button>
                </div>
            </form>
        </div>
    </div>
</body>
</html>

```

save_passwords.html

```

{% extends 'manager/base.html' %}

{% block content %}
<div class="row justify-content-center">
    <div class="col-md-6">
        <div class="p-5 border" style="border-radius: 24px; background-color: #ffffff;">
            <h3 class="fw-bold mb-4 text-center">🔒 Зберегти пароль</h3>

            <form method="POST">
                {% csrf_token %}
                <div class="mb-3">
                    <label class="form-label fw-bold text-muted">Назва сервісу (напр. Instagram)</label>
                    <input type="text" name="service_name" class="form-control form-control-lg" required>
                </div>

                <div class="mb-3">
                    <label class="form-label fw-bold text-muted">Ваш логін або Email</label>

```

```

        <input type="text" name="login" class="form-control form-control-lg" required>
    </div>

    <div class="mb-4">
        <label class="form-label fw-bold text-muted">Пароль</label>
        <input type="text" name="password" class="form-control form-control-lg" value="{{ pwd }}" required>
    </div>

    <button type="submit" class="btn btn-primary btn-lg w-100 rounded-pill fw-bold">Зберіть у
Сховище</button>
    </form>
</div>
</div>
{% endblock %}

```

admin.py

```

from django.contrib import admin
from .models import Category, PasswordRecord

admin.site.register(Category)
admin.site.register>PasswordRecord

```

apps.py

```

from django.apps import AppConfig

class ManagerConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'manager'

```

models.py

```

from django.db import models
from django.contrib.auth.models import User

class Category(models.Model):
    name = models.CharField(max_length=100, verbose_name="Назва категорії")
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='categories')

    def __str__(self):
        return f'{self.name} ({self.user.username})'

class PasswordRecord(models.Model):
    service_name = models.CharField(max_length=255, verbose_name="Назва сервісу (напр. Google)")
    login = models.CharField(max_length=255, verbose_name="Логін або Email")
    encrypted_password = models.CharField(max_length=500, verbose_name="Зашифрований пароль")

    category = models.ForeignKey(Category, on_delete=models.SET_NULL, null=True, blank=True,
    verbose_name="Категорія")
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='passwords',
    verbose_name="Власник")
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.service_name} - {self.login}'

```

views.py

```

from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth import update_session_auth_hash

def register_view(request):
    error_message = None
    if request.method == 'POST':
        user_name = request.POST.get('username')
        email = request.POST.get('email')
        pass1 = request.POST.get('password')
        pass2 = request.POST.get('password_confirm')

        if pass1 != pass2:
            error_message = "Паролі не співпадають!"
        elif User.objects.filter(username=user_name).exists():
            error_message = "Користувач з таким логіном вже існує!"
        else:
            user = User.objects.create_user(username=user_name, email=email, password=pass1)
            user.save()
            login(request, user)
            return redirect('home')

    return render(request, 'manager/register.html', {'error': error_message})

def login_view(request):
    error_message = None
    if request.method == 'POST':
        user_name = request.POST.get('username')
        pass1 = request.POST.get('password')

        user = authenticate(request, username=user_name, password=pass1)

        if user is not None:
            login(request, user)
            return redirect('home')
        else:
            error_message = "Невірний логін або пароль!"

    return render(request, 'manager/login.html', {'error': error_message})

def home_view(request):
    return render(request, 'manager/home.html')

def logout_view(request):
    logout(request)
    return redirect('login')

from django.contrib.auth.decorators import login_required
from .models import PasswordRecord

@login_required(login_url='login')
def save_password_view(request):
    generated_pwd = request.GET.get('pwd', "")

    if request.method == 'POST':
        service = request.POST.get('service_name')
        login_val = request.POST.get('login')
        password_val = request.POST.get('password')

        PasswordRecord.objects.create(
            user=request.user,
            service_name=service,
            login=login_val,
            encrypted_password=password_val

```

```

    )
    return redirect('passwords')

return render(request, 'manager/save_password.html', {'pwd': generated_pwd})

@login_required(login_url='login')
def passwords_view(request):
    records = PasswordRecord.objects.filter(user=request.user).order_by('-created_at')
    return render(request, 'manager/passwords.html', {'records': records})
@login_required(login_url='login')
def check_view(request):
    return render(request, 'manager/check.html')

from django.contrib.auth import update_session_auth_hash

@login_required(login_url='login')
def profile_view(request):
    saved_count = PasswordRecord.objects.filter(user=request.user).count()
    success_msg = None
    error_msg = None

    if request.method == 'POST':
        if 'update_profile' in request.POST:
            new_username = request.POST.get('username')
            new_email = request.POST.get('email')

            if User.objects.filter(username=new_username).exclude(id=request.user.id).exists():
                error_msg = "Цей логін вже зайнятий іншим користувачем!"
            else:
                request.user.username = new_username
                request.user.email = new_email
                request.user.save()
                success_msg = "Особисті дані успішно оновлено!"

        elif 'change_password' in request.POST:
            new_pass1 = request.POST.get('new_password')
            new_pass2 = request.POST.get('confirm_password')

            if new_pass1 == new_pass2:
                request.user.set_password(new_pass1)
                request.user.save()
                update_session_auth_hash(request, request.user)
                success_msg = "Пароль від акаунта успішно змінено!"
            else:
                error_msg = "Нові паролі не співпадають!"

    return render(request, 'manager/profile.html', {
        'saved_count': saved_count,
        'success_msg': success_msg,
        'error_msg': error_msg
    })

def about_view(request):
    return render(request, 'manager/about.html')
def custom_admin_view(request):
    if not request.user.is_superuser:
        return redirect('home')

    success_msg = None

    if request.method == 'POST':
        user_id = request.POST.get('user_id')
        action = request.POST.get('action')
        target_user = User.objects.get(id=user_id)

```

```

if action == 'update_email':
    target_user.email = request.POST.get('new_email')
    target_user.save()
    success_msg = f"Пошту для користувача {target_user.username} оновлено!"

elif action == 'update_password':
    target_user.set_password(request.POST.get('new_password'))
    target_user.save()
    success_msg = f"Пароль для користувача {target_user.username} успішно змінено!"

users = User.objects.exclude(id=request.user.id).order_by('-date_joined')

return render(request, 'manager/custom_admin.html', {
    'users': users,
    'success_msg': success_msg
})

```

manage.py

```

import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

```